2026年04月21日(火)
3,4,5限 ＠研A302

プロジェクト演習 テーマD
第2回

担当：CS学部 教授 伏見卓恭
連絡先：fushimity@edu.teu.ac.jp

授業の流れ

- 第1回：実験環境の構築／Gitの復習／Pygameの基礎
- 第2回：関数を用いたゲーム開発の基礎／コード規約とコードレビュー
- 第3回：オブジェクト指向によるゲーム開発／ブランチとマージによる開発
- 第4回：Pygameらしいゲーム開発／フォークとプルリクによる共同開発
- 第5回：共同開発演習（ベース部分＋個別機能実装）
- 第6回：共同開発演習（統合と調整）
- 第7回：共同開発演習（成果発表）

本日のお品書き

1. 前回の復習
2. リーダブルコード
   - コメント，docstring
   - 型ヒント，関数アノテーション
   - コード規約
3. Pygameの演習
4. コードレビュー

目標：Pygameの理解を深め，読みやすいコードを実装でき，
GitHubでIssueの送受信ができる

前回の復習

配布物の確認

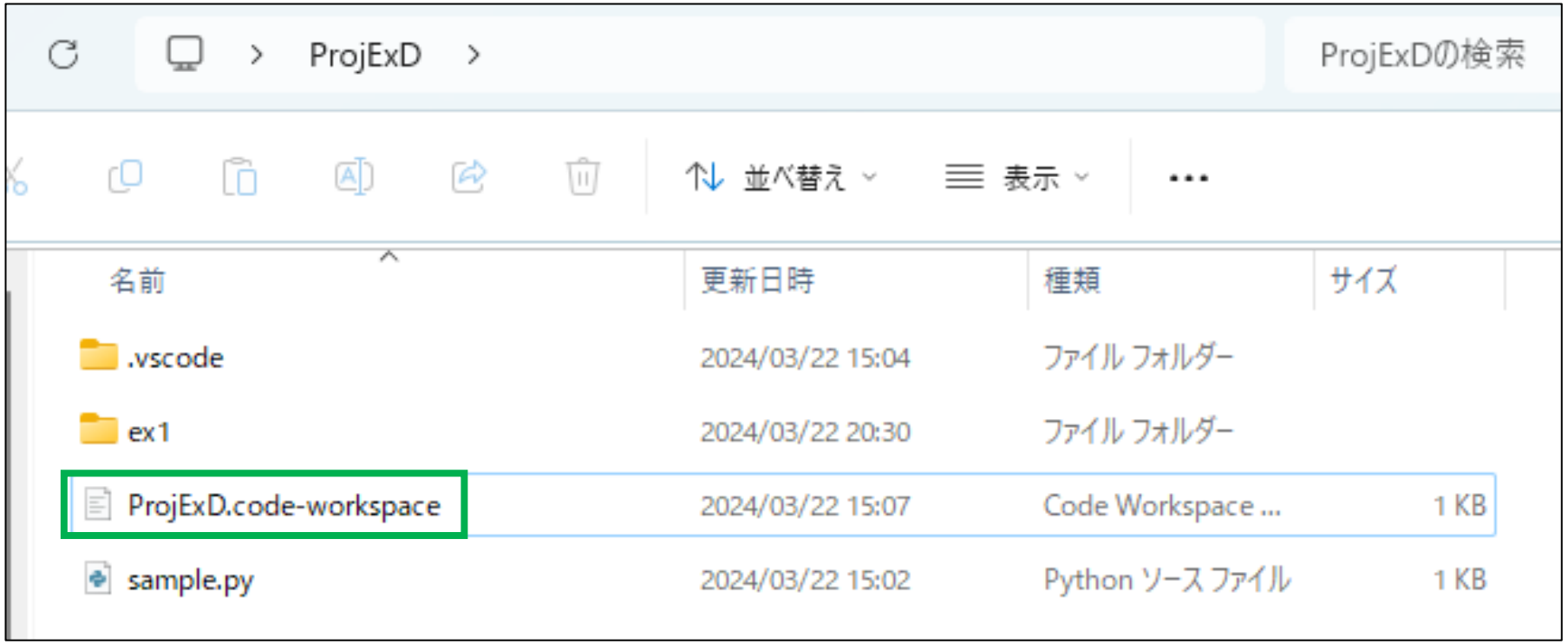
- ex2.zipをProjExDフォルダにDLし，展開する

【配布物配置後のディレクトリ構造】

- ProjExD/
  - sample.py
  - ex1/
  - ex2/　　本日の配布物
    - dodge_bomb.py　・・・逃げろ！こうかとん
    - fig/
      - pg_bg.jpg　・・・背景画像
      - {0, …, 9}.png　・・・こうかとん画像

VScodeの起動

- ProjExDフォルダ内の「ProjExD.code-workspace」をダブルクリックし，ワークスペースを開く



前回の復習：git

- 「ProjExD¥ex2」フォルダを右クリックし，Git Bashを起動する
- gitリポジトリを初期化する：`git init`
- 全ファイルをステージングする：`git add .`
- 「初期状態」というコメントでコミットする：`git commit -m "初期状態"`

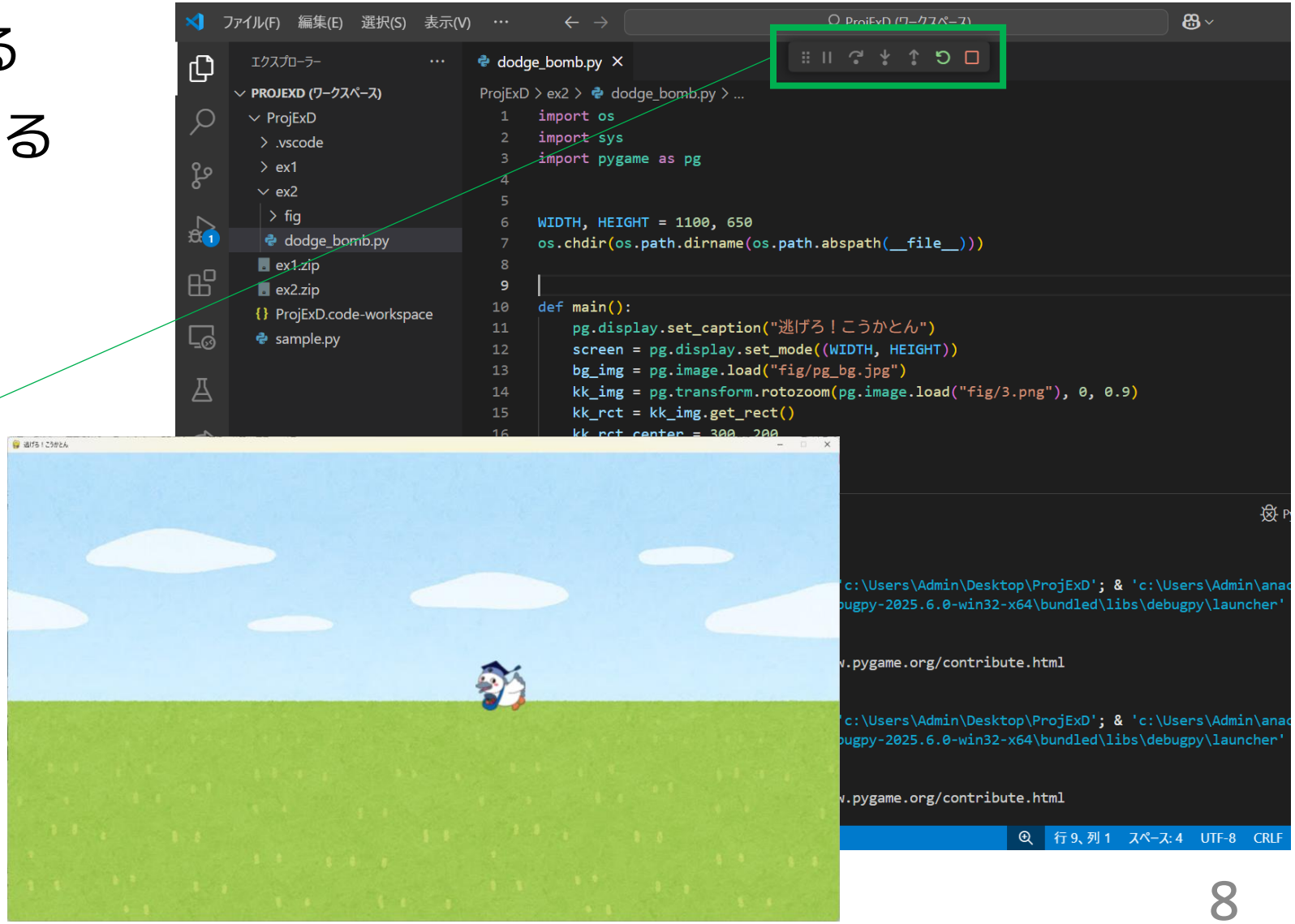
- GitHubに「ProjExD_2」という公開リポジトリを作成する
- 公開リポジトリの情報を「origin」という名前で登録する：
  `git remote add origin https://github.com/自分のアカウント/ProjExD_2.git`
- 公開リポジトリにコミット履歴をプッシュする：`git push origin main`

Git for Windowsでない人は
SSH版をコピペ

まずは，dodge_bomb.pyを動かしてみる

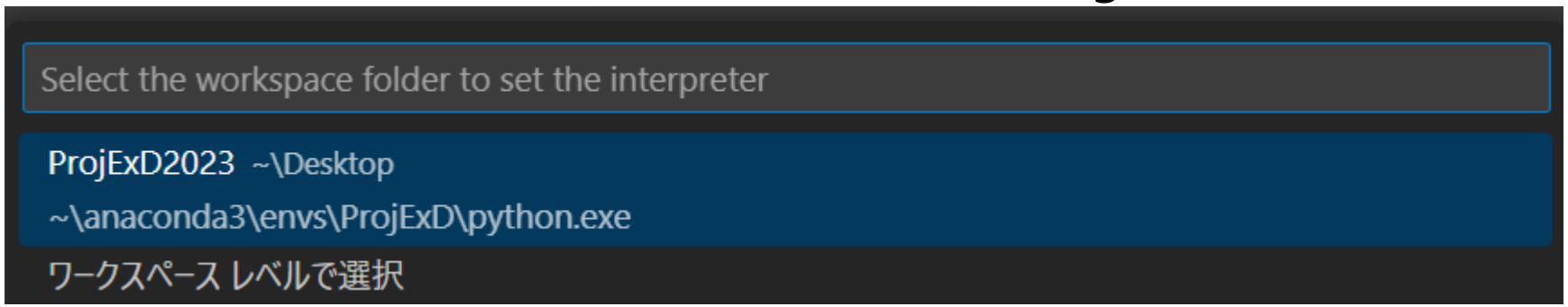
- 「Ctrl＋S」で保存する
- 「Ctrl＋F5」で実行する

- 「□」で終了する

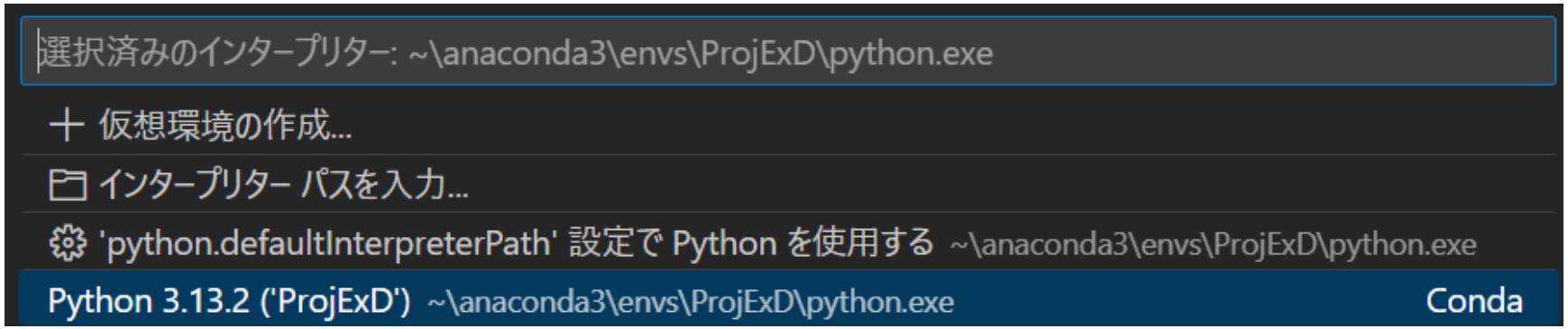


実行できなかったら → VScode右下を確認

- Python 3.13.2 ('ProjExD': conda) となっておらず，
  「Select…」またはPythonのバージョンだけになっている場合は，
  クリックして，
- ワークスペースフォルダとして「ProjExD」を選ぶ



- インタープリターパスとして「Python3.13.2 ('ProjExD')」を選ぶ



前回の復習：dodge_bomb.py

【ゲームの初期化】

- ウィンドウタイトルを「逃げろ！こうかとん」とする
- 幅1100×高さ650のスクリーンSurfaceを生成する
- 背景画像pg_bg.jpgのSurfaceを生成する
- こうかとん画像3.pngを0.9倍に拡大したSurfaceを生成する
- こうかとんSurfaceに対応するRectを取得し，初期座標300，200を設定する

【ゲームのループ】

- 背景SurfaceをスクリーンSurfaceに貼り付ける
- 押下キーを取得し，キーに応じて合計移動量を計算する
- こうかとんSurfaceをRectの設定に従い，スクリーンSurfaceに貼り付ける
- 画面を更新する

3限：リーダブルコード

コメント

Pythonでは，複数行のコメントを書く構文はないが，
ダブルクォーテーション3つで複数行の文字列を作ることができる．
単なる文字列に対しては何もしないため，コメントとして機能する．

コメント

```
print()  # 行末コメント
# 1行のコメント



"""
複数行の文字列
複数行の文字列
複数行の文字列
"""
```

← 「#」の後は半角スペース1つ入れる
← 行末の場合は「#」の前に半角スペース2つ入れる
← インデントの深さはコードと合わせる
← 長いコメントは行末（インラインコメント）ではなく，
　独立した行コメントにする
← 何をやっているか明らかなことは書かない

docstring

モジュールやクラス，関数に関する説明の複数行コメントのことで，
モジュールの先頭，クラス定義の直後，関数定義の直後に書かれる．
docstringの内容は特殊属性__doc__に格納される．
help()関数により，docstringの内容を確認することもできる．

docstring

```
def hoge():
    """
    複数行の文字列
    複数行の文字列
    """

print(hoge.__doc__)
```

```
def read_names(file_path: str):
    """
    poke_names.txtを読み込む関数
    引数：ファイルのパス
    戻り値：名前文字列のリスト，タイプリストのリスト，…
    """

print(read_names.__doc__)
```

← help(read_names)でもOK

```
read_names(file_path: str)
    poke_names.txtを読み込む関数
    引数：ファイルのパス
    戻り値：名前文字列のリスト，タイプリストのリスト，進化先リストのリスト
```

型ヒント，関数アノテーション

- Pythonのオブジェクトは，型(type)，値(value)，同一性(id)の3要素からなり，型を明示的に示すことを型ヒントという
- 関数定義部で用いる場合が多い → 関数アノテーションと呼ぶ
- あくまでヒント，アノテーションであり，エラーを出したりはしない

型ヒントの例

```
# 宣言と代入を同時に行う場合
level: int = 51

# 宣言と代入を別々に行う場合
level: int
level = 51
```

明らかな場合は省略しても問題ない

関数アノテーションの例

```
def __init__(self, name: str, types: str) -> None:
```

docstringと関数アノテーションの使用例

```python
def check_bound(obj_rct: pg.Rect) -> tuple[bool, bool]:
    """
    引数：こうかとんRectかばくだんRect
    戻り値：タプル（横方向判定結果，縦方向判定結果）
    画面内ならTrue，画面外ならFalse
    """
    yoko, tate = True, True
    if obj_rct.left < 0 or WIDTH < obj_rct.right: # 横方向判定
        yoko = False
    if obj_rct.top < 0 or HEIGHT < obj_rct.bottom: # 縦方向判定
        tate = False
    return yoko, tate
```

docstringや
関数アノテーション
を書いておくと，

関数使用時に，VScodeが
ヒントを表示してくれる

```
check
 # 練 check_bound
```

```
def check_bound(obj_rct: Rect) -> tuple ×
```

引数：こうかとんRectかばくだんRect 戻り値：タプル（横方向判定結果，縦方向判定結果） 画面内ならTrue，画面外ならFalse

コード規約

- コード規約：https://pep8-ja.readthedocs.io/ja/latest/
- わかりやすい記事：https://qiita.com/simonritchie/items/bb06a7521ae6560738a7

「一貫性にこだわりすぎるのは、狭い心の現れである」
つまり，規約に囚われすぎない臨機応変さも必要

コード規約（つづき）

以下，抜粋

- 文中改行
  - 要素の先頭を合わせる
  - 1つインデントを入れる
  - 引数の場合は2つインデントを入れる
- 空行の数
  - トップレベルは2行入れる
  - それ以外は1行入れる
- import文
  - カンマで区切って1行にまとめず，1つずつ書く
    ただし，同じモジュールから複数の関数やクラスをimportするときは，1行にまとめる
  - 標準ライブラリ→サードパーティー→自作の順
  - 基本はアルファベット順

コード規約（つづき）

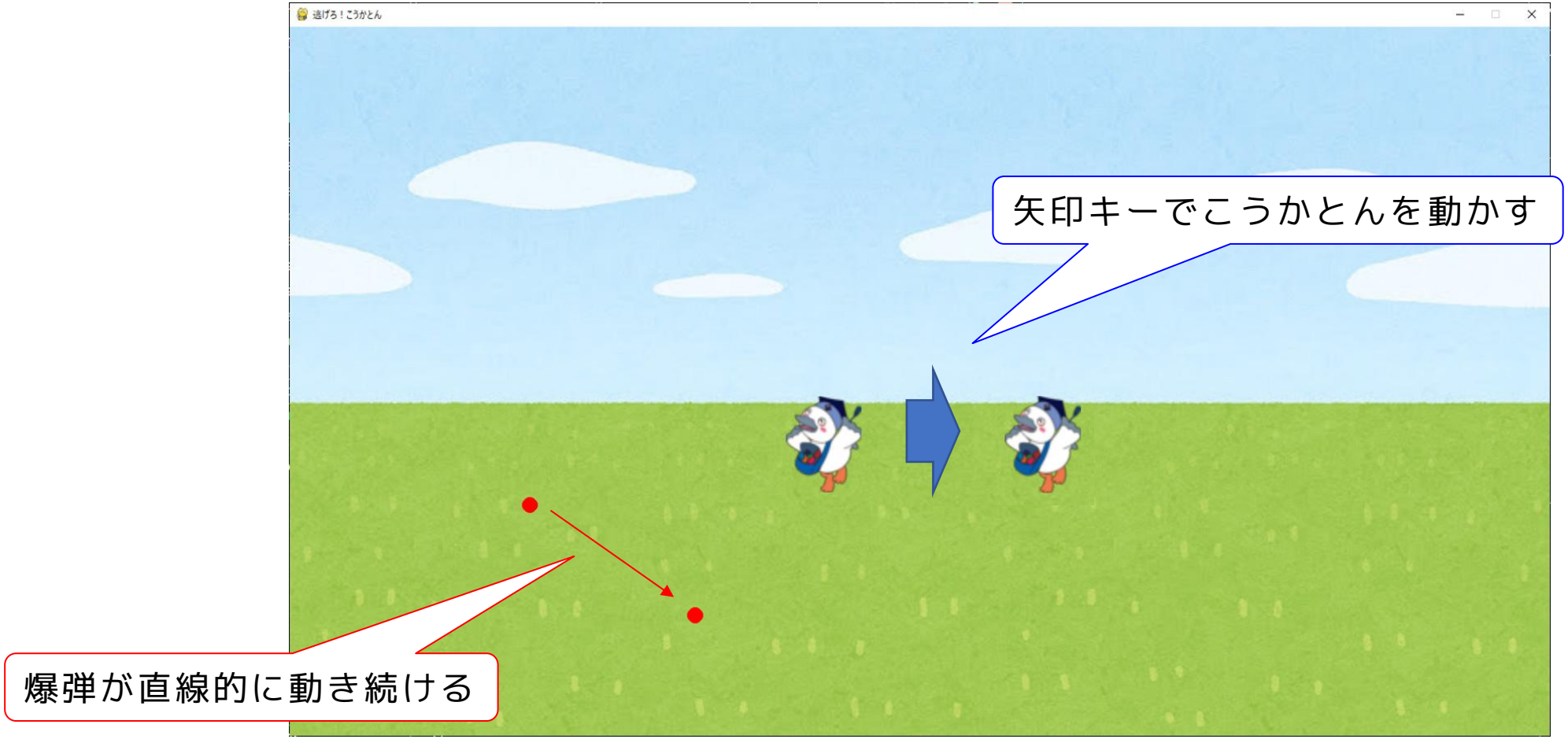
- 空白
  - 余分な空白は入れない
  - 統一する（カンマの後ろ，演算子の前後）
  - 「＝」の位置を合わせるために空白を入れない
  - キーワード引数，デフォルト引数の「＝」前後は空白を入れない
- 命名規則
  - 意味を表した単語にする（よく知られた省略は可能）
  - クラス名：パスカルケース（例：SampleClass）
  - 関数名，変数名：スネークケース（例：sample_func）
  - 定数名：全て大文字でアンダースコアでつなげる（例：SAMPLE_CONST）
  - 予約語などと被る場合：後ろにアンダースコアを付ける（例：id_）
- 条件文
  - Noneとの比較：「==」は使わず「is None」，「is not None」
  - 真理値との比較：「==」は使わない
    - ＮＧな例：if key_lst[pg.K_UP] == True, if key_lst[pg.K_UP] != True
    - ＯＫな例：if key_lst[pg.K_UP], if not key_lst[pg.K_UP]

練習問題（Pygameの基本）

dodge_bomb.py

「逃げろ！こうかとん」を実装しよう

- 野原で遊ぶこうかとんに爆弾が襲い掛かる．
  爆弾から逃げるゲームを実装する．



練習問題

※1問ずつステージング，コミットしよう

1. 押下キーに応じてこうかとんを動かす部分のコードにおいて，移動量辞書を定義することで，冗長な記述を改めよ
   - 押下キーと移動量の対応関係を表す辞書DELTAを定義する

辞書のキー：押下キー	辞書の値：移動量タプル
pg.K_UP ← 上矢印	(0, -5) ← 上に5
pg.K_DOWN ← 下矢印	(0, +5) ← 下に5
pg.K_LEFT ← 左矢印	(-5, 0) ← 左に5
pg.K_RIGHT ← 右矢印	(+5, 0) ← 右に5

練習問題

※1問ずつステージング，コミットしよう

2. 爆弾Surface bb_img を作成し，ランダムな位置に配置し，whileが回るたびに爆弾を移動，表示せよ

- 半径：10／色：赤　の円　←第1回講義資料P.54を参考にする
- 爆弾Surfaceの黒い部分を透明にするには「set_colorkey(黒)」を利用する
- 爆弾Rect bb_rct の位置を表す変数に乱数を設定する
- ためしにwhileループの中でblitして，表示されるか確認する　←第1回P.55

- 横方向速度：vx = +5／縦方向速度：vy = +5　※ v はvelocity（速度）の意味
- 爆弾Rectのmove_ip(vx, vy)メソッドで速度に応じて位置を移動させる

↑ 第1回P.56

練習問題

※1問ずつステージング，コミットしよう

3．こうかとんと爆弾が画面の外に出ないようにせよ．

- 画面内or画面外の判定をするcheck_bound関数を実装する
  - 引数：こうかとんRect or 爆弾Rect
  - 戻り値：横方向・縦方向の真理値タプル（True：画面内／False：画面外）
  - Rectオブジェクトのleft，right，top，bottomの値から画面内・外を判断する
- 更新後の座標が画面外になった場合の挙動
  - こうかとん：更新前の位置に戻す
  - 爆弾：横（縦）方向に出そうになったらvx（vy）の符号を反転する

4．こうかとんが爆弾と衝突したらmain関数からreturnするようにせよ．

- 判定にはRectクラスのcolliderect()を使用する ←第1回P.56

4限：演習問題

演習課題：「逃げろ！こうかとん」の改良

- 以下の追加機能を実装せよ（どの機能から実装しても構わない）
  1. ゲームオーバー画面（こうかとん画像の切替と「GameOver」表示）
  2. 時間とともに爆弾が拡大，加速する
  3. 飛ぶ方向に従ってこうかとん画像を切り替える
  4. 追従型爆弾（爆弾がこうかとんに近づくように移動する）
  5. その他，独自の機能

例：`git commit -m "追加機能１途中"`

- 注意事項
  - 関数を定義して実装すること（何でも関数にすれば良いわけではないが，練習ということで）
  - 型ヒントとdocstringを導入すると，点数が高くなる
  - どの追加機能を実装したのかわかるように，コミットコメントに機能番号を入れること（採点時に必要）
  - 頻繁にコミットする場合は，完成なのか途中なのかわかるようにすること

直前のコミットコメントの修正方法

- 原則：プッシュにより公開した後は修正しない
- 1つ前（＝直前）のコミットコメントを修正する場合

```
git commit --amend -m "修正後のコメント"
```

または

```
git commit --amend
```

←エディタでコメントを修正する

（Advancedな内容なので，TASA教員に聞いてみよう）

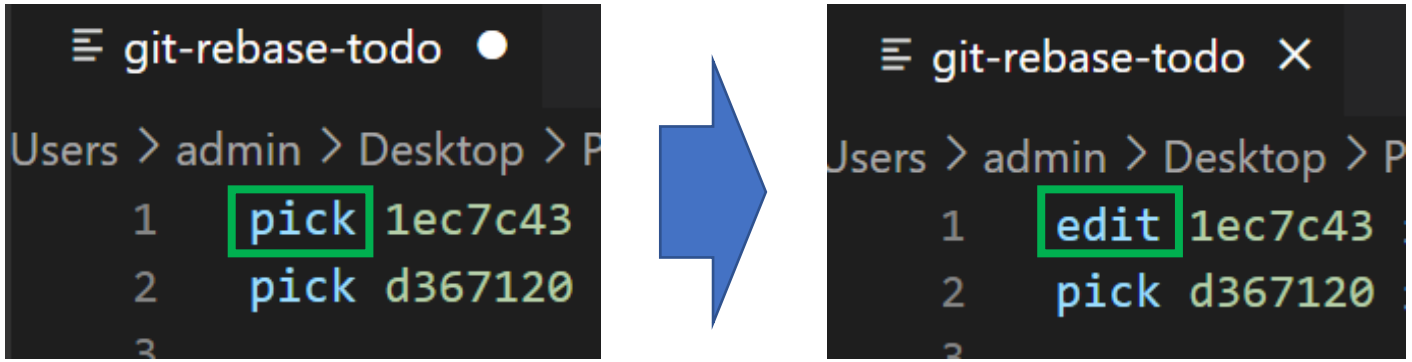
直前以外のコミットコメントの修正方法

- 原則：プッシュにより公開した後は修正しない
- 直前以外のコミットコメントを修正する場合

  `git rebase -i HEAD~2` ← 2つ前の場合

  `hint: Waiting for your editor to close the file...`と表示され，裏でエディタが開いているはずなので，見てみる

  - `pick` を `edit` に書き換え，保存して，閉じる　※2つ前に戻っている状態



  - 前ページを参照して，amendでコメントを修正する：`git commit --amend`
  - リベースを完了する：`git rebase --continue`

コーディング時に意識してみよう

- 読みやすさ：空白，空行，文中改行の入れ方
- 簡潔さ↔冗長さ，短さ，一貫性
- 変数名，関数名，クラス名
- 一時変数の利用
- 全体の構造：クラス，関数を定義しているか？
- ループの書き方：`for i in range:` → `for x in lst:`
- 条件文の書き方：`if hoge == True:` → `if hoge:`
- ネストの深さが深すぎないか？
- 必要十分なコメントや型ヒント，docstringがあるか？
- 修正容易性，拡張容易性

1.ゲームオーバー画面（こうかとん画像の切替と「Game Over」表示）

- こうかとんに爆弾が着弾した際に，
  - 画面をブラックアウトし，
  - 泣いているこうかとん画像と
  - 「Game Over」の文字列を
  - 5秒間表示させる関数を実装する
- 関数の実装例
  - 定義： `def gameover(screen: pg.Surface) -> None`
  - 中身：
    1. 黒い矩形を描画するための空のSurfaceを作り，黒い矩形を描画する ← 第1回資料P.54参照
    2. 1のSurfaceの透明度を設定する ← ココ参照
    3. 白文字でGame Overと書かれたフォントSurfaceを作り，1のSurfaceにblitする ← 第1回資料 P.53
    4. こうかとん画像をロードし，こうかとんSurfaceを作り，1のSurfaceにblitする
    5. 1のSurfaceをscreen Surfaceにblitする
    6. pg.display.update()したら，time.sleep(5)する
  - 使い方：こうかとんと爆弾が衝突したら呼び出す

2.時間とともに爆弾が拡大，加速する

- 関数の実装例
  - 定義：
    ```
    def init_bb_imgs() -> tuple[list[pg.Surface], list[int]]
    ```
  - 中身：
    - 無限に拡大，加速するのはおかしいので， 10段階程度の大きさを変えた爆弾Surfaceのリストと加速度のリストを準備する
    - 爆弾Surfaceのリスト（一部掲載）：
      ```
      for r in range(1, 11):
          bb_img = pg.Surface((20*r, 20*r))
          pg.draw.circle(bb_img, (255, 0, 0), (10*r, 10*r), 10*r)
          bb_imgs.append(bb_img)
      ```
    - 加速度のリスト：`bb_accs = [a for a in range(1, 11)]`
    - これらのリストのタプルbb_imgs, bb_accsを返す
  - 使い方：
    - while文の前で呼び出して2つのリストを得る／while文の中でtmrの値に応じて，リストから適切な要素を選択する（一部掲載）：
      ```
      avx = vx*bb_accs[min(tmr//500, 9)]
      bb_img = bb_imgs[min(tmr//500, 9)]
      ```
      ← このavxとavyをmove_ipメソッドに渡す
    - Surfaceの大きさが変わった場合は，Rectのwidth属性とheight属性を更新する（一部掲載）：
      ```
      bb_rct.width = bb_img.get_rect().width
      ```

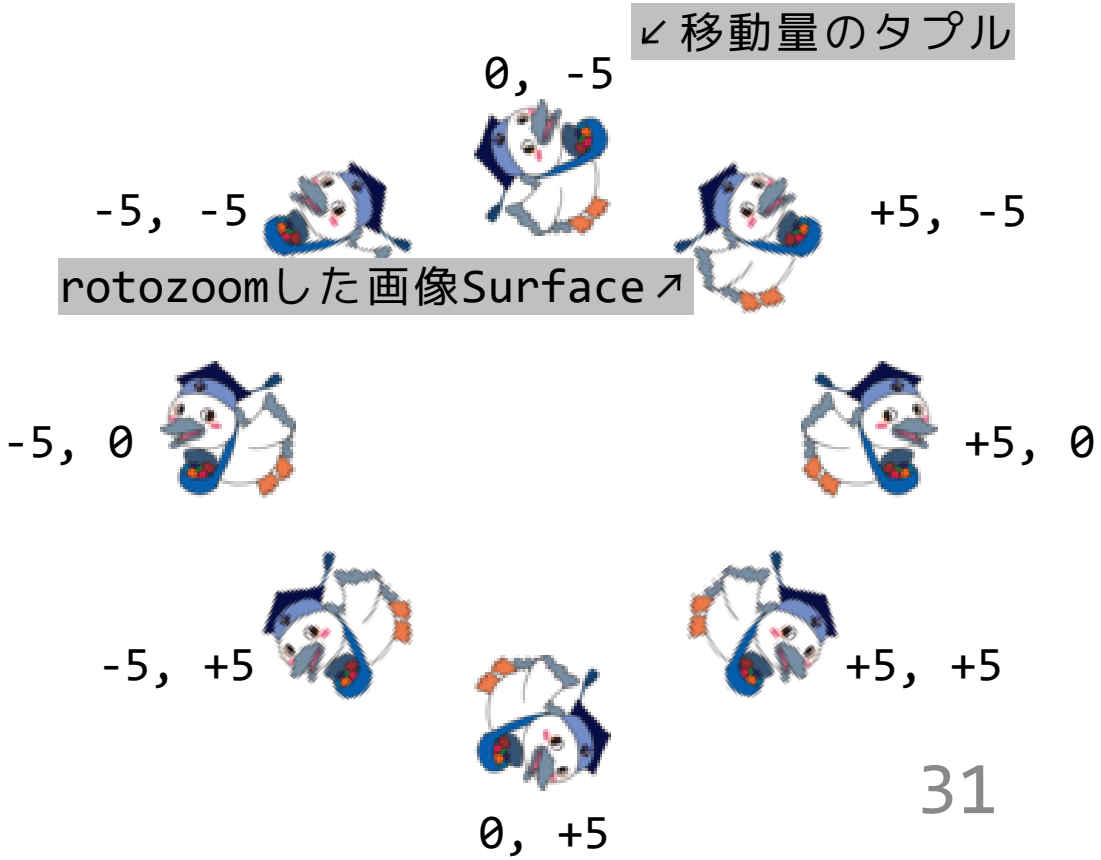
3.飛ぶ方向に従ってこうかとん画像を切り替える

- 移動量タプルと対応する画像Surfaceの辞書を返す関数を実装する
- 関数の実装例
  - 定義： `get_kk_imgs() -> dict[tuple[int, int], pg.Surface]`
  - 中身：
    - 押下キーに対する移動量の合計値タプルをキー，rotozoomしたSurfaceを値とした辞書を返す

```
kk_dict = {
    ( 0, 0): rotozoom(???)  # キー押下がない場合
    (+5, 0): rotozoom(???)  # 右
    (+5,-5): rotozoom(???)  # 右上
    ( 0,-5): rotozoom(???)  # 上
    ...                     ↑ こうかとんの回転は
}                             第1回資料P.52を参照
```

  - 使い方：
    - while文の前で呼び出して辞書を得る
    - while文の中で，合計移動量をタプル化して上記辞書のキーとして値を取得する

```
kk_img = kk_imgs[tuple(sum_mv)]
```



4.追従型爆弾（爆弾がこうかとんに近づくように移動する）

- 爆弾から見て，こうかとんRectがある方向，すなわち移動すべき方向をベクトル(vx，vy)を返す関数を実装する
- 関数の実装例
  - 定義：

```
def calc_orientation(org: pg.Rect, dst: pg.Rect, current_xy: tuple[float, float])
                                                                -> tuple[float, float]
```

  - 中身：
    - org（爆弾を想定）とdst（こうかとんを想定）の座標ベクトル間の差ベクトルを求める
    - 差ベクトルのノルムが√50になるように正規化する
      - 正規化しないと，一瞬でこうかとんに追いついてしまう
      - move_ipが小数点以下を無視するため，位置関係によっては爆弾が水平or垂直に移動してしまう（気にしなくていい）
    - ただし，爆弾とこうかとんの距離（＝正規化前の差ベクトルのノルム）が300未満だったら，慣性としてcurrent_xy（計算前の方向）に移動させる
      - すぐにゲームオーバーにならないようにするため
    - 正規化後方向ベクトル or 計算前の方向ベクトルを返す

速度ベクトルのノルム：√(5^2+5^2)

  - 使い方：while文の中で呼び出してベクトル（vx，vyのタプル）を取得する

5限：コードレビュー

4限演習課題のコミットをGitHubにプッシュせよ：`git push origin main`

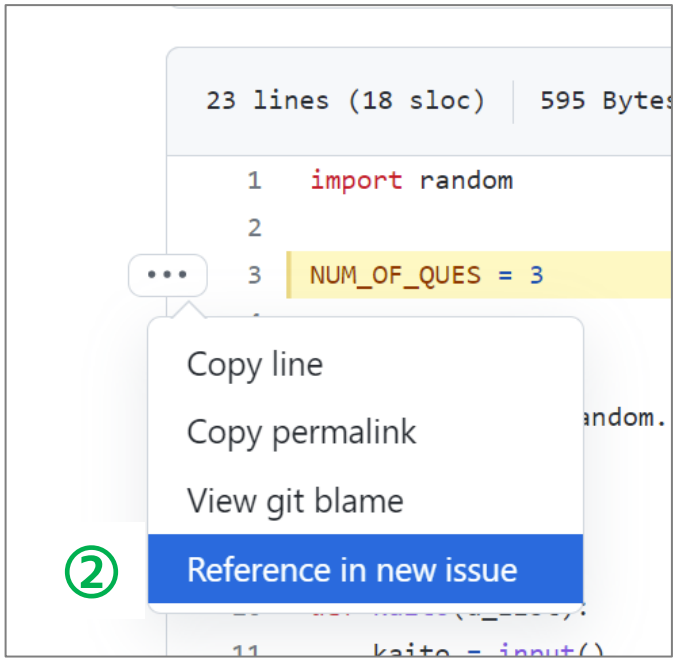
コードレビューの手順

※実装途中やバグありの場合は
`git checkout コミットID`でバグなしの状態まで遡ってみよう
（Advancedな内容なので，TASA教員に聞いてみよう）

1. グループメンバー1人の公開リポジトリURLを入手する
   - 各自，最低1人のコードをレビューし，最低1人にレビューされることになる
2. コードを見て，リーダブルコードやコード規約の観点で修正すべき点を挙げる
3. 次ページを参照し，Issueにより修正すべき点を１つずつ送信する
   - 修正すべき点がない完璧なコードの場合，他のメンバーにレビューを依頼する
   - それでも修正すべき点がない場合，TASA，教員にレビューを依頼する
4. Issueを受信したら，必要に応じてコードを修正しプッシュする

Issueを送る手順

1. グループメンバーのGitHubのページにアクセスし，当該コードを開く
   https://github.com/fushimity/ProjExD_2/blob/main/dodge_bomb.py
2. コードの該当行を選択し，「・・・」→「Reference in new issue」をクリックする
3. タイトルとコメントを書く
   - 対象が複数行の場合，手動で行数を追加する：「#L13」→「#L13-L16」



Issueを送る手順

4. Previewで確認する
5. 「Submit new issue」をクリックし送信する

Issueが届く

- Issuesタブから，Issueコメントを読み，対応する

コード修正後の手順

1. 受信したIssueから1つ選び，コメントに従ってコードを修正する
2. 修正が終わったらステージングする：`git add ファイル名`
3. ステージングされた内容をコミットする
   ※重要：コミットコメントに，対応したIssue番号を付けること
   `git commit -m "コメント #番号"`
   例：`git commit -m "変数名統一 #1"`
   ・「#」の前に半角スペースが必要
   ・「#番号」は半角で入力する
   ・間違えた場合は，P.26を参考にしてamendで修正する

   これにより，Issueと対応コミットがGitHub上で紐づけられる

4. リモートリポジトリにプッシュする：`git push origin main`

チェック項目

1. 追加機能（3点満点x最大4機能）
   - 追加機能が正常に動いている：+1点
   - 関数を用いて実装している：+1点
   - 追加実装した関数に型ヒントとdocstringの両方がある：+1点

2. 全体がリーダブルコードか？
   - 空白，空行が適切か：+1点
   - 必要十分なコメント：+1点

3. Issue番号を付してコミット：+1点

4. 提出物不備（ファイル名，クリッカブル，内容物）は1点ずつ減点

提出物

学籍番号は，半角・大文字で

- ファイル名：C0A25XXX_kadai2.pdf
- 内容：以下の順番でPDFを結合して提出すること
  - コミット履歴
    https://github.com/fushimity/ProjExD_2/commits/main
  - コードの最終版
    https://github.com/fushimity/ProjExD_2/blob/main/dodge_bomb.py
  - Issue一覧
    https://github.com/fushimity/ProjExD_2/issues

自分のアカウント名

提出物の作り方

※スクショ画像は認めません．
以下の手順に従ってPDFを作成し，提出すること

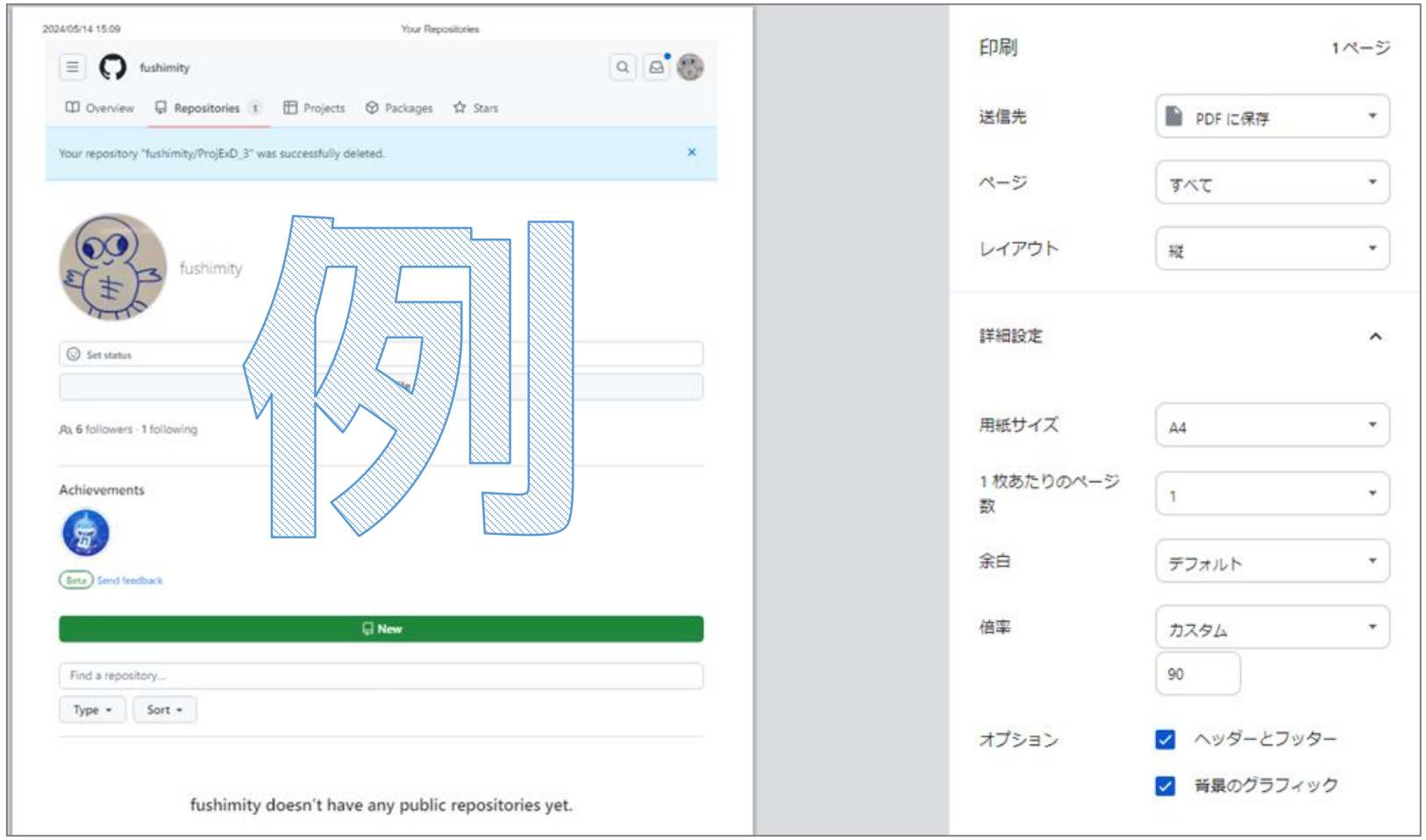
1. ChromeでPDFとして保存する（次ページを参照）

2. 以下のURLから各PDFをマージする
   https://www.ilovepdf.com/ja/merge_pdf

3. ファイル名を「C0A25XXX_kadai2.pdf」として保存する

ChromeでPDFとして保存する方法

1. 該当ページを表示させた状態で「Ctrl＋P」
2. 以下のように設定し，「保存」をクリックする



Microsoft Print to PDFはダメ

←送信先：PDFに保存

←ページ：すべて

←レイアウト：縦

←用紙サイズ：A4

←余白：デフォルト

←倍率：90

←両方チェック

チェックの手順

※基本的に，TASAチェックは１度だけですので，
　慎重に提出物PDFを作ること．どうしてもの場合は要相談．

1．受講生：提出物（pdf）を作成する
2．受講生：担当TASAに提出物と成果物（ゲーム）を見せに行く
3．TASA：提出物とゲームのデモを確認し，点数を確定する
4．受講生：提出物をMoodleにアップロードして，帰る
5．FSM：近日中に課題と点数を確認し，Moodleに登録する

- 時間内にチェックが終わらなそうな場合は，
  提出物をMoodleに提出してから帰る
  （次回までor次回の3限にチェックされる）

← 授業後に提出した場合
時間外提出扱いになり
割引いて採点するので
できるだけ提出して，
チェックを受けること